Ansible — Interview Ready Answers

Comprehensive answers with examples for all 7 Ansible questions

7 Questions • Playbooks • Roles • Inventory • Vault • Idempotency • CI/CD Integration • Large-Scale Automation

Q1. Describe your experience with Ansible or configuration management tools and how you've used them in production.

Answer

This is an experience-based question. The goal is to demonstrate practical, production-level knowledge of Ansible — not just theoretical understanding. Below is a comprehensive overview of how Ansible is used across real DevOps environments, which you can adapt to reflect your own experience.

What is Ansible and Why It Matters

Ansible is an open-source, agentless configuration management, provisioning, and orchestration tool developed by Red Hat. It uses YAML-based playbooks to describe infrastructure state declaratively. Unlike Puppet or Chef, Ansible requires no agents on managed nodes — it communicates over SSH (or WinRM for Windows), making adoption fast and secure.

Feature	Ansible Advantage
Agentless	No software installed on managed nodes — SSH only
YAML syntax	Human-readable playbooks — low learning curve
Idempotent	Safe to run multiple times — same result each time
Extensible	Thousands of built-in modules + custom module support
Push-based	Control node pushes configuration to targets — no polling
Inventory	Static files or dynamic inventory from AWS, Azure, GCP APIs

Production Use Cases

1. Server Provisioning and Configuration

Ansible was used to bootstrap new EC2 instances immediately after Terraform provisioned them. The pipeline flow was: Terraform applies (infrastructure) → Ansible playbook runs (OS configuration, packages, agents):

```
# Example: Configure a new web server
---
- name: Configure web server
  hosts: web_servers
  become: true
  roles:
    - common          # Base OS hardening, NTP, logging agents
    - nginx           # Install and configure NGINX
    - app_deploy      # Deploy application from S3 artifact
```

2. Application Deployment

Ansible handled zero-downtime rolling deployments to EC2 fleets before EKS was adopted. Using serial to deploy to batches of servers sequentially:

```
- name: Rolling deployment
  hosts: app_servers
  serial: 2          # Deploy to 2 servers at a time
  tasks:
    - name: Take server out of load balancer
      uri:
        url: 'https://alb-api/deregister/{{ inventory_hostname }}'
        method: POST
    - name: Deploy new application version
      unarchive:
        src: 's3://artifacts/app-{{ version }}.tar.gz'
        dest: /opt/app/
        remote_src: yes
    - name: Re-register with load balancer
      uri:
```

```
      url: 'https://alb-api/register/{{ inventory_hostname }}'
      method: POST
```

3. Compliance and Security Hardening

Ansible was used to enforce CIS benchmark controls across hundreds of servers — patching, firewall rules, SSH configuration, and service hardening. Running nightly to detect and remediate configuration drift:

```
# Nightly compliance playbook
ansible-playbook cis-hardening.yml \
  --check --diff          # Dry-run first
  -i inventory/production/
```

4. Multi-Environment Infrastructure Management

Separate inventories per environment (dev/staging/prod) with environment-specific group variables. The same playbooks run against all environments — only the variables change.

5. Integration with CI/CD

Ansible playbooks triggered by Jenkins or GitHub Actions after a Terraform apply completes. The pipeline passes Terraform outputs (IP addresses, instance IDs) to Ansible as extra variables:

```
- name: Run Ansible after Terraform
  run: |
    terraform output -json > tf_outputs.json
    ansible-playbook site.yml \
      -i inventory/ \
      --extra-vars @tf_outputs.json
```

Ansible vs Other Configuration Management Tools

Tool	Architecture	Language
Ansible	Agentless (SSH/WinRM)	YAML
Puppet	Agent-based (pull)	Puppet DSL
Chef	Agent-based (pull)	Ruby DSL
SaltStack	Agent-based or SSH	YAML + Jinja
Terraform	Agentless (API)	HCL

> *Tip: When answering this in an interview, relate each use case to a specific project outcome: 'We reduced server provisioning time from 2 hours of manual steps to 8 minutes using Ansible' or 'Ansible compliance playbooks cut our CIS benchmark violations from 40% to 3% across 200 servers.'*

Q2. How do you use Ansible for automated server provisioning in a CI/CD pipeline?

Answer

Integrating Ansible into CI/CD pipelines creates a fully automated path from infrastructure provisioning to application-ready servers. Ansible bridges the gap between Terraform (which creates the infrastructure) and the running application.

The Complete Provisioning Pipeline

```
Git push → CI/CD triggers → Terraform apply (creates EC2/VM)
→ Dynamic inventory discovers new instances
→ Ansible playbook runs (OS config, packages, app deploy)
→ Health check verifies server is ready
→ Server added to load balancer
```

Step 1 — Ansible in GitHub Actions / Jenkins

```
# .github/workflows/provision.yml
name: Provision Servers
on:
  push:
    branches: [main]

jobs:
  provision:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v4

    - name: Install Ansible
```

```
      run: pip install ansible boto3 botocore

    - name: Terraform Apply
      run: |
        cd terraform/
        terraform init && terraform apply -auto-approve
        terraform output -json > ../ansible/tf_outputs.json

    - name: Wait for SSH to be ready
      run: |
        ansible all -i ansible/inventory/aws_ec2.yml \
          -m wait_for_connection \
          --timeout 120

    - name: Run Ansible Provisioning Playbook
      run: |
        ansible-playbook ansible/site.yml \
          -i ansible/inventory/aws_ec2.yml \
          --extra-vars '@ansible/tf_outputs.json' \
          --private-key $SSH_PRIVATE_KEY_FILE \
          -e env=production
      env:
        ANSIBLE_HOST_KEY_CHECKING: 'False'
        AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
        AWS_SECRET_ACCESS_KEY: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
```

Step 2 — Dynamic Inventory (AWS EC2 Plugin)

Instead of maintaining a static inventory file, use the AWS EC2 dynamic inventory plugin to automatically discover new instances by their tags:

```
# ansible/inventory/aws_ec2.yml
plugin: amazon.aws.aws_ec2
regions:
  - us-east-1
filters:
  tag:Environment: production
  tag:ManagedBy: terraform
  instance-state-name: running
keyed_groups:
  - prefix: role
    key: tags.Role          # Groups: role_webserver, role_database
  - prefix: env
    key: tags.Environment   # Groups: env_production, env_staging
hostnames:
  - private-ip-address     # Use private IPs for VPC access
```

When Terraform creates a new EC2 instance with tag Role=webserver, it automatically appears in the role_webserver group for Ansible — no manual inventory update needed.

Step 3 — The Provisioning Playbook

```
# ansible/site.yml
---
- name: Common configuration for all servers
  hosts: all
  become: true
  roles:
    - common

- name: Configure web servers
  hosts: role_webserver
  become: true
  roles:
    - nginx
    - app_deploy
    - monitoring_agent

- name: Configure database servers
  hosts: role_database
  become: true
  roles:
    - postgresql
    - backup_agent
```

Step 4 — The Common Role Tasks

```
# roles/common/tasks/main.yml
---
- name: Update all packages
  ansible.builtin.dnf:
    name: '*'
    state: latest
    update_cache: yes

- name: Install required packages
  ansible.builtin.package:
    name:
      - amazon-ssm-agent
      - cloudwatch-agent
      - htop
      - curl
    state: present

- name: Start and enable SSM agent
  ansible.builtin.systemd:
    name: amazon-ssm-agent
    state: started
    enabled: yes

- name: Configure NTP
  ansible.builtin.template:
    src: chrony.conf.j2
    dest: /etc/chrony.conf
  notify: restart chrony

- name: Set hostname from inventory
  ansible.builtin.hostname:
    name: '{{ inventory_hostname }}'
```

Step 5 — Post-Provisioning Verification

```
- name: Verify web server is responding
  ansible.builtin.uri:
    url: 'http://{{ inventory_hostname }}/health'
    status_code: 200
  register: health_check
  retries: 5
  delay: 10
  until: health_check.status == 200
```

Passing Terraform Outputs to Ansible

When Terraform creates infrastructure, it produces outputs (RDS endpoint, ALB DNS, etc.) that Ansible needs for configuration. Pass them as extra variables:

```
# terraform outputs.tf
output "rds_endpoint" { value = aws_db_instance.main.endpoint }
output "alb_dns"      { value = aws_lb.main.dns_name }

# In CI/CD pipeline
terraform output -json > tf_outputs.json
ansible-playbook site.yml --extra-vars '@tf_outputs.json'

# In playbook — use the Terraform output
- name: Configure application database connection
  ansible.builtin.template:
    src: app.conf.j2
    dest: /etc/app/config.conf
  vars:
    db_host: '{{ rds_endpoint }}'
```

> *Tip: Use --check --diff before the real run in CI/CD pipelines to show what changes will be made. This acts as a dry-run preview — engineers can review before applying. Only proceed to the real apply after review.*

Q3. What's your approach to managing Ansible roles and inventories across multiple environments?

Answer

Managing multiple environments with Ansible requires a structured approach to directory layout, inventory organisation, variable precedence, and role reuse. The goal is to share the same playbooks and roles across all environments, with only the configuration differing per environment.

Recommended Directory Structure

```
ansible/
├── site.yml                    # Master playbook — entry point
├── ansible.cfg                 # Global Ansible configuration
│
├── inventory/
│   ├── dev/
│   │   ├── hosts.yml           # Static hosts for dev (or aws_ec2.yml)
│   │   ├── group_vars/
│   │   │   ├── all.yml         # Variables for ALL hosts in dev
│   │   │   ├── web_servers.yml # Variables for web_servers group in dev
│   │   │   └── db_servers.yml
│   │   └── host_vars/
│   │       └── app01.yml       # Variables for a specific host
│   ├── staging/
│   │   ├── hosts.yml
│   │   └── group_vars/
│   │       └── all.yml
│   └── production/
│       ├── aws_ec2.yml         # Dynamic inventory for prod
│       └── group_vars/
│           ├── all.yml
│           └── web_servers.yml
│
├── roles/
│   ├── common/                 # Applied to all servers
│   ├── nginx/                  # NGINX web server
│   ├── postgresql/             # PostgreSQL database
│   ├── app_deploy/             # Application deployment
│   └── monitoring_agent/       # CloudWatch / Prometheus agent
│
├── playbooks/
│   ├── deploy_app.yml
│   ├── patch_servers.yml
│   └── rotate_certs.yml
│
└── requirements.yml            # External roles from Ansible Galaxy
```

Inventory Design — Static vs Dynamic

Dev environments often use static inventory for simplicity. Production uses dynamic inventory (AWS EC2 plugin, Azure RM plugin) to automatically track server changes:

```
# inventory/dev/hosts.yml — static
all:
  children:
    web_servers:
      hosts:
        dev-web-01:
          ansible_host: 10.0.1.10
        dev-web-02:
          ansible_host: 10.0.1.11
    db_servers:
      hosts:
        dev-db-01:
          ansible_host: 10.0.2.10

# inventory/production/aws_ec2.yml — dynamic
plugin: amazon.aws.aws_ec2
regions: [us-east-1, eu-west-1]
filters:
  tag:Environment: production
  instance-state-name: running
keyed_groups:
  - key: tags.Role
    prefix: role
```

Group Variables — Environment-Specific Configuration

The same variable name holds different values per environment. Ansible loads group_vars automatically based on the inventory directory used:

```
# inventory/dev/group_vars/all.yml
env: dev
app_version: latest
db_host: dev-postgres.internal
log_level: debug
nginx_worker_processes: 2

# inventory/production/group_vars/all.yml
env: production
app_version: '{{ lookup("env", "RELEASE_VERSION") }}'
db_host: prod-postgres.internal
log_level: warn
nginx_worker_processes: auto
```

Run against a specific environment by pointing to its inventory directory:

```
ansible-playbook site.yml -i inventory/dev/
ansible-playbook site.yml -i inventory/staging/
ansible-playbook site.yml -i inventory/production/
```

Variable Precedence (Lowest to Highest)

Ansible has 22 levels of variable precedence. The most important to understand for multi-environment management:

Precedence	Location
Low	role defaults/main.yml
Medium	inventory group_vars/all.yml
Medium-high	inventory group_vars/<group>.yml
High	playbook vars:
Higher	--extra-vars (-e)
Highest	--extra-vars (JSON)

Role Management

Internal Roles

Roles specific to your organisation live in the roles/ directory. Each role follows the standard structure:

```
roles/nginx/
├── tasks/
│   ├── main.yml        # Entry point — includes other task files
│   ├── install.yml
│   └── configure.yml
├── handlers/
│   └── main.yml        # Restart nginx on config change
├── templates/
│   └── nginx.conf.j2   # Jinja2 template for nginx config
├── files/
│   └── default.html
├── vars/
│   └── main.yml        # Role-internal variables (high precedence)
├── defaults/
│   └── main.yml        # Safe default values (lowest precedence)
└── meta/
    └── main.yml        # Role dependencies, galaxy metadata
```

External Roles — Ansible Galaxy

Use requirements.yml to declare external roles and install them in CI/CD:

```
# requirements.yml
roles:
  - name: geerlingguy.docker
    version: 6.1.0
  - name: geerlingguy.nodejs
    version: 6.1.0

collections:
  - name: amazon.aws
    version: '>=6.0.0'
  - name: community.postgresql
```

```
    version: '>=3.0.0'

# Install in CI pipeline
ansible-galaxy install -r requirements.yml
ansible-galaxy collection install -r requirements.yml
```

ansible.cfg — Standardise Behaviour

```
[defaults]
inventory          = inventory/
roles_path         = roles/:~/.ansible/roles
remote_user        = ec2-user
private_key_file   = ~/.ssh/ansible-key.pem
host_key_checking  = False
retry_files_enabled = False
stdout_callback    = yaml

[ssh_connection]
pipelining         = True    # Reduces SSH connections — faster
ssh_args           = -o ControlMaster=auto -o ControlPersist=60s
```

> *Tip: Use ansible-lint in your CI/CD pipeline to enforce code quality standards on playbooks and roles. It catches common mistakes, deprecated syntax, and style inconsistencies before they reach production.*

Q4. How do you handle secrets and sensitive data in Ansible (Ansible Vault)?

Answer

Ansible Vault encrypts sensitive data — passwords, API keys, private keys, certificates — so they can be safely stored in version control alongside the rest of your Ansible code. Without Vault, sensitive data must be kept out of Git entirely, creating a management nightmare.

What Ansible Vault Does

Ansible Vault uses AES-256 encryption to encrypt any YAML file or specific string values within YAML files. The encrypted content is unreadable without the vault password. Ansible automatically decrypts at runtime when the password is provided.

Method 1 — Encrypt an Entire File

Best for files that are entirely sensitive — vars files containing multiple secrets:

```
# Encrypt an existing file
ansible-vault encrypt inventory/production/group_vars/all/vault.yml

# Create a new encrypted file
ansible-vault create inventory/production/group_vars/all/vault.yml

# Edit an encrypted file
ansible-vault edit inventory/production/group_vars/all/vault.yml

# View without editing
ansible-vault view inventory/production/group_vars/all/vault.yml

# Decrypt to plaintext (use carefully — don't commit decrypted files)
ansible-vault decrypt inventory/production/group_vars/all/vault.yml
```

Vault File Content Structure

Keep a non-encrypted vars file alongside the encrypted vault file. Non-sensitive variables in the plain file, sensitive values in vault.yml:

```
# group_vars/all/vars.yml  — NOT encrypted (safe to commit)
env: production
app_port: 8080
db_host: prod-postgres.internal
db_user: app_user
db_password: '{{ vault_db_password }}'     # References vault variable
api_key: '{{ vault_api_key }}'

# group_vars/all/vault.yml  — ENCRYPTED with ansible-vault
vault_db_password: 'S3cur3P@ss!word'
vault_api_key: 'sk-prod-abc123xyz'
vault_ssl_cert: |
```

```
  -----BEGIN CERTIFICATE-----
  MIIFxxxxxx...
  -----END CERTIFICATE-----
```

By convention, vault variables are prefixed with vault_ to distinguish them from regular variables. The regular vars file references vault variables, so playbooks always use the clean name (db_password) not the vault name (vault_db_password).

Method 2 — Encrypt Individual Strings (Inline Vault)

Encrypt specific values within a YAML file, keeping other values readable:

```
# Generate an encrypted string
ansible-vault encrypt_string 'S3cur3P@ss!word' --name 'db_password'

# Output — paste this into your vars file:
db_password: !vault |
          $ANSIBLE_VAULT;1.1;AES256
          383232313864396231623033303937643965663135613436343932 61
          343238653834633639346564363839323839 37...

# Your vars file looks like this:
env: production             # Plaintext — readable
db_host: prod-db.internal   # Plaintext — readable
db_password: !vault |       # Encrypted — unreadable without password
          $ANSIBLE_VAULT;1.1;AES256
          383232313864396231623033303937643965 66...
```

Running Playbooks with Vault

```
# Method 1: Interactive prompt (development)
ansible-playbook site.yml -i inventory/prod/ --ask-vault-pass

# Method 2: Password file (CI/CD)
echo $VAULT_PASSWORD > /tmp/vault_pass.txt
chmod 600 /tmp/vault_pass.txt
ansible-playbook site.yml -i inventory/prod/ --vault-password-file /tmp/vault_pass.txt
rm /tmp/vault_pass.txt   # Clean up immediately

# Method 3: Environment variable (CI/CD — GitHub Actions)
export ANSIBLE_VAULT_PASSWORD_FILE=/tmp/vault_pass

# Method 4: Python script as password provider (HashiCorp Vault integration)
ansible-playbook site.yml --vault-password-file vault_pass_script.py
```

Multiple Vault Passwords — Vault IDs

In large organisations with different teams managing different secrets, use Vault IDs to separate passwords per secret category:

```
# Encrypt with a specific vault ID
ansible-vault encrypt_string 'prod_pass' --vault-id prod@prompt --name db_password
ansible-vault encrypt_string 'dev_pass' --vault-id dev@prompt --name db_password

# Run with multiple vault passwords
ansible-playbook site.yml \
  --vault-id prod@/secrets/prod_vault_pass \
  --vault-id dev@/secrets/dev_vault_pass
```

Integration with External Secret Managers

For production environments, integrate Ansible Vault with AWS Secrets Manager or HashiCorp Vault via a custom vault password script. The script fetches the vault password at runtime — no password file stored anywhere:

```
#!/usr/bin/env python3
# vault_pass_script.py — fetches vault password from AWS Secrets Manager
import boto3, sys

client = boto3.client('secretsmanager', region_name='us-east-1')
response = client.get_secret_value(SecretId='ansible/vault-password')
print(response['SecretString'], end='')

# Use it:
ansible-playbook site.yml --vault-password-file vault_pass_script.py
```

Best Practices

Practice	Reason

Never commit plaintext secrets	Git history is permanent — rotation doesn't help once committed
Use vault_ prefix for encrypted vars	Clearly distinguishes secret vars from regular vars
Keep vault password in CI/CD secret store	GitHub Actions Secrets, Jenkins Credentials — never in code
Rotate vault password periodically	Re-encrypt all vault files with new password quarterly
Use ansible-lint to catch unencrypted secrets	Detects patterns that look like secrets in plaintext
Separate vault files per environment	Dev and prod have different vault passwords for isolation

> *Tip: Use ansible-vault rekey to change the vault password when rotating credentials. Run: ansible-vault rekey inventory/production/group_vars/all/vault.yml — it prompts for the old password and new password, re-encrypts in place.*

Q5. How do you ensure idempotency in Ansible playbooks?

Answer

Idempotency is the property where running a playbook multiple times produces the same result as running it once. It means the second run makes no changes if the system is already in the desired state. Idempotency is fundamental to safe, reliable automation — it allows you to run playbooks for remediation or updates without fear of breaking things that are already correct.

Why Idempotency Matters

- Safety — A playbook can be run during incidents, scheduled jobs, or pipeline retries without causing unintended side effects.
- Remediation — Nightly playbook runs detect and fix configuration drift automatically.
- Pipeline reliability — If a pipeline reruns a failed step, idempotent tasks don't redo completed work.
- Audit and compliance — Playbooks can be run read-only (--check) to verify current state without making changes.

1. Use Ansible's Built-in Idempotent Modules

The vast majority of Ansible modules are idempotent by design. They check the current state before making changes and only act if needed:

```
# IDEMPOTENT — package module checks if nginx is already installed
- name: Install NGINX
  ansible.builtin.package:
    name: nginx
    state: present

# IDEMPOTENT — file module checks if file/directory exists and has correct attributes
- name: Create application directory
  ansible.builtin.file:
    path: /opt/myapp
    state: directory
    owner: appuser
    mode: '0755'

# IDEMPOTENT — user module checks if user already exists
- name: Create application user
  ansible.builtin.user:
    name: appuser
    shell: /bin/bash
    groups: sudo
    state: present

# IDEMPOTENT — service module checks current state before changing
- name: Ensure nginx is running
  ansible.builtin.service:
    name: nginx
    state: started
    enabled: yes
```

2. Use creates and removes for Command/Shell Tasks

The command and shell modules are NOT idempotent by default — they run every time. Use creates or removes to make them conditional:

```
# NOT IDEMPOTENT — runs every time
- name: Extract application archive
  ansible.builtin.command:
    cmd: tar -xzf /tmp/app.tar.gz -C /opt/app/

# IDEMPOTENT — only runs if the extracted directory doesn't exist
- name: Extract application archive
  ansible.builtin.command:
    cmd: tar -xzf /tmp/app.tar.gz -C /opt/app/
    creates: /opt/app/app_binary    # Skip if this file already exists

# IDEMPOTENT — only runs if the file exists (then deletes it)
- name: Remove temporary setup file
  ansible.builtin.command:
    cmd: rm -f /tmp/setup_flag
    removes: /tmp/setup_flag        # Skip if this file doesn't exist
```

3. Use when Conditions with Registered Variables

Check current state before running a potentially non-idempotent task:

```
- name: Check if application is already configured
  ansible.builtin.stat:
    path: /opt/app/.configured
  register: app_configured

- name: Run initial setup (only if not already done)
  ansible.builtin.script:
    cmd: /opt/app/setup.sh
  when: not app_configured.stat.exists

- name: Create marker file after setup
  ansible.builtin.file:
    path: /opt/app/.configured
    state: touch
  when: not app_configured.stat.exists
```

4. Use the lineinfile and blockinfile Modules

For configuration file management, avoid replacing entire files with copy when only specific lines need to change. Use lineinfile or blockinfile — they check before making changes:

```
# IDEMPOTENT — adds line only if not already present
- name: Add custom sysctl setting
  ansible.builtin.lineinfile:
    path: /etc/sysctl.conf
    line: 'net.ipv4.ip_forward = 1'
    regexp: '^net.ipv4.ip_forward'
    state: present

# IDEMPOTENT — replaces the matching block if different, adds if missing
- name: Configure application block in config file
  ansible.builtin.blockinfile:
    path: /etc/app/config.conf
    marker: '# {mark} ANSIBLE MANAGED BLOCK'
    block: |
      [database]
      host = {{ db_host }}
      port = 5432
      name = {{ db_name }}
```

5. Templates are Naturally Idempotent

The template module computes the template, compares with the existing file content (using checksums), and only replaces if different:

```
- name: Deploy nginx configuration
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    owner: root
    mode: '0644'
    validate: nginx -t -c %s    # Validate before replacing
  notify: restart nginx         # Handler runs only if file changed
```

6. Handlers — Run Only When Triggered

Handlers are tasks that run only when notified by a task that actually made a change. If the template didn't change, the restart handler doesn't run:

```
# In tasks:
- name: Deploy nginx config
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
  notify: restart nginx   # Notifies handler only if config changed

# In handlers/main.yml:
- name: restart nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
```

On first run: template changes → handler notified → nginx restarts. On second run: template unchanged → handler NOT notified → nginx NOT restarted. This is idempotent service management.

7. Avoid Non-Idempotent Patterns

Non-Idempotent Pattern	Problem
shell: echo 'text' >> file	Appends every run — duplicate entries
command: mkdir /opt/dir	Fails on second run if dir exists
command: useradd myuser	Fails on second run if user exists
shell: pip install package	Reinstalls every run
command: ./setup.sh	Runs every time

8. Test Idempotency with --check

```
# First run — makes changes
ansible-playbook site.yml -i inventory/prod/

# Second run — verify no changes (idempotency check)
ansible-playbook site.yml -i inventory/prod/ --check
# All tasks should show 'ok' not 'changed'
# Any 'changed' on second run indicates a non-idempotent task
```

Tip: In CI/CD, run the playbook twice sequentially and assert that the second run reports zero changed tasks. This automated idempotency test catches regressions when new tasks are added — and it's the gold standard for verifying playbook quality.

Q6. How do you structure Ansible code for large-scale multi-environment deployments?

Answer

At scale — hundreds of servers, dozens of roles, multiple teams, and 3+ environments — Ansible code structure determines whether the codebase is maintainable or a liability. The following structure has been battle-tested for large production deployments.

The Complete Enterprise Ansible Project Structure

```
ansible/
├── ansible.cfg                     # Project-wide configuration
├── site.yml                        # Master playbook (imports all others)
├── requirements.yml                # External role/collection dependencies
│
├── inventory/                      # All inventories
│   ├── dev/
│   │   ├── hosts.yml
│   │   └── group_vars/
│   │       ├── all/
│   │       │   ├── vars.yml        # Non-sensitive vars
│   │       │   └── vault.yml       # Encrypted sensitive vars
│   │       └── web_servers.yml
│   ├── staging/
│   └── production/
```

```
│       ├── aws_ec2.yml              # Dynamic inventory
│       └── group_vars/
│           ├── all/
│           │   ├── vars.yml
│           │   └── vault.yml
│           ├── web_servers.yml
│           └── database_servers.yml
│
├── roles/                           # All roles
│   ├── common/                      # Base OS — applied to ALL servers
│   ├── security_hardening/          # CIS benchmarks
│   ├── monitoring/                  # Agent installation
│   ├── nginx/
│   ├── postgresql/
│   └── application/                 # App deployment role
│
├── playbooks/                       # Targeted, purpose-specific plays
│   ├── deploy.yml
│   ├── patch.yml
│   ├── backup.yml
│   └── rollback.yml
│
├── filter_plugins/                  # Custom Jinja2 filters
├── library/                         # Custom Ansible modules
└── tests/                           # Molecule tests for roles
    ├── common/
    └── nginx/
```

Master Playbook — site.yml

The master playbook imports purpose-specific playbooks and applies roles to groups. Keep it clean and high-level — it's the entry point everyone reads:

```
# site.yml
---
# Apply base configuration to ALL servers
- name: Common configuration
  import_playbook: playbooks/common.yml

# Role-specific configuration
- name: Configure web servers
  import_playbook: playbooks/web_servers.yml

- name: Configure database servers
  import_playbook: playbooks/database_servers.yml

- name: Deploy application
  import_playbook: playbooks/deploy.yml
```

Playbook Files — Single Responsibility

```
# playbooks/web_servers.yml
---
- name: Configure web servers
  hosts: web_servers
  become: true
  vars_files:
    - '{{ inventory_dir }}/group_vars/web_servers.yml'
  roles:
    - role: common
    - role: security_hardening
    - role: monitoring
    - role: nginx
      vars:
        nginx_ssl_enabled: '{{ enable_ssl | default(true) }}'
    - role: application
```

Role Design for Scale

Roles must be self-contained and parameterisable. Never hardcode environment-specific values inside a role — accept all variable values through defaults:

```
# roles/nginx/defaults/main.yml
---
nginx_user: www-data
nginx_worker_processes: auto
```

```
nginx_worker_connections: 1024
nginx_keepalive_timeout: 65
nginx_ssl_enabled: false
nginx_ssl_cert_path: ''
nginx_listen_port: 80
nginx_server_name: '_'
nginx_document_root: /var/www/html
nginx_log_dir: /var/log/nginx

# roles/nginx/tasks/main.yml
---
- name: Install NGINX
  ansible.builtin.package:
    name: nginx
    state: present

- name: Deploy NGINX configuration
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    validate: 'nginx -t -c %s'
  notify: reload nginx

- name: Include SSL tasks
  ansible.builtin.include_tasks: ssl.yml
  when: nginx_ssl_enabled | bool
```

Jinja2 Templates for Dynamic Configuration

Templates generate environment-specific configuration files from variables. One template serves all environments:

```
# roles/nginx/templates/nginx.conf.j2
user {{ nginx_user }};
worker_processes {{ nginx_worker_processes }};

events {
    worker_connections {{ nginx_worker_connections }};
}

http {
    keepalive_timeout {{ nginx_keepalive_timeout }};

    {% if nginx_ssl_enabled %}
    ssl_certificate     {{ nginx_ssl_cert_path }};
    ssl_certificate_key {{ nginx_ssl_key_path }};
    ssl_protocols       TLSv1.2 TLSv1.3;
    {% endif %}

    {% for upstream in nginx_upstreams %}
    upstream {{ upstream.name }} {
        {% for server in upstream.servers %}
        server {{ server }};
        {% endfor %}
    }
    {% endfor %}
}
```

Role Testing with Molecule

Molecule is the standard testing framework for Ansible roles. It spins up containers or VMs, runs the role, verifies the result, and tests idempotency automatically:

```
# Install molecule
pip install molecule molecule-docker

# Initialise molecule for a role
cd roles/nginx/
molecule init scenario

# Run full test sequence
molecule test
# This runs: create → converge → idempotency check → verify → destroy

# molecule/default/verify.yml
```

```
---
- name: Verify nginx
  hosts: all
  tasks:
  - name: Check nginx is running
    ansible.builtin.service_facts:
  - name: Assert nginx is running
    assert:
      that:
        - "'nginx' in services"
        - "services['nginx'].state == 'running'"
  - name: Check nginx is listening on port 80
    ansible.builtin.wait_for:
      port: 80
      timeout: 10
```

Performance Optimisations for Large Scale

- SSH Multiplexing — Reuse SSH connections with ControlMaster to avoid reconnecting for every task:

```
[ssh_connection]
ssh_args = -o ControlMaster=auto -o ControlPersist=60s -o StrictHostKeyChecking=no
pipelining = True
```

- Forks — Run tasks on more hosts in parallel (default is 5):

```
[defaults]
forks = 20    # Run on 20 hosts simultaneously
```

- strategy: free — Hosts proceed independently through tasks (default is linear where all hosts wait at each task):

```
- hosts: web_servers
  strategy: free    # Fast hosts don't wait for slow ones
```

- async Tasks — Long-running tasks run in the background:

```
- name: Run long database migration
  ansible.builtin.command: /opt/app/migrate.sh
  async: 600      # Allow up to 600 seconds
  poll: 30        # Check progress every 30 seconds
```

Scale	Configuration
< 50 servers	forks=5 (default)
50-200 servers	forks=15, pipelining=True
200-500 servers	forks=25, strategy=free
500+ servers	AWX/Tower, forks=50+, callback plugins

> *Tip: For organisations managing 200+ servers, invest in Red Hat Ansible Automation Platform (formerly Ansible Tower) or its open-source version AWX. They provide a web UI, RBAC, job scheduling, audit trails, and API access — making large-scale Ansible operations manageable for teams.*

Q7. How would you use Ansible to configure 100 servers simultaneously without SSH key issues?

Answer

Configuring 100 servers simultaneously with Ansible requires solving three distinct challenges: efficient SSH connectivity at scale, SSH key distribution and trust, and parallelism tuning. Each has a proven solution.

Challenge 1 — SSH Key Distribution to New Servers

Before Ansible can manage a server, it needs SSH access. On freshly provisioned servers, only the cloud provider's default key pair is installed. There are several approaches:

Approach A — Cloud Provider Key Pair + Ansible Key Rotation

Use the cloud-provisioned key pair for initial access, then use Ansible itself to add the Ansible service account's public key:

```
# bootstrap.yml — Run ONCE with the cloud key pair
---
- name: Bootstrap servers with Ansible service account
  hosts: all
  remote_user: ec2-user               # Cloud default user
  vars:
    ansible_ssh_private_key_file: /tmp/cloud-key.pem
  tasks:
```

```
    - name: Create ansible service account
      ansible.builtin.user:
        name: ansible
        shell: /bin/bash
        groups: wheel
      become: true

    - name: Deploy Ansible SSH public key
      ansible.posix.authorized_key:
        user: ansible
        key: '{{ lookup("file", "/etc/ansible/id_rsa.pub") }}'
        state: present
      become: true

    - name: Configure sudo without password for ansible
      ansible.builtin.lineinfile:
        path: /etc/sudoers.d/ansible
        line: 'ansible ALL=(ALL) NOPASSWD:ALL'
        create: yes
        validate: 'visudo -cf %s'
      become: true

# After bootstrap, all subsequent playbooks use the ansible account
ansible-playbook site.yml -i inventory/prod/ \
  --user ansible \
  --private-key /etc/ansible/id_rsa
```

Approach B — Cloud-Init / User Data (Best for Scalability)

Inject the Ansible public key during instance provisioning via cloud-init. No bootstrap playbook needed — every instance is SSH-ready the moment it starts:

```
# Terraform — inject SSH key via user_data
resource "aws_instance" "app_server" {
  ami           = var.ami_id
  instance_type = "t3.medium"
  user_data     = <<-EOF
    #!/bin/bash
    useradd -m -s /bin/bash ansible
    mkdir -p /home/ansible/.ssh
    echo "${var.ansible_public_key}" >> /home/ansible/.ssh/authorized_keys
    chown -R ansible:ansible /home/ansible/.ssh
    chmod 700 /home/ansible/.ssh
    chmod 600 /home/ansible/.ssh/authorized_keys
    echo "ansible ALL=(ALL) NOPASSWD:ALL" > /etc/sudoers.d/ansible
  EOF
}
```

Approach C — AWS Systems Manager (No SSH at All)

For AWS environments, Ansible can connect via SSM Session Manager instead of SSH. This eliminates all SSH key management — no keys to distribute, no port 22 to open:

```
# Install SSM plugin for SSH proxy
# ~/.ssh/config
Host i-* mi-*
  ProxyCommand sh -c 'aws ssm start-session --target %h --document-name AWS-StartSSHSession
--parameters portNumber=%p'
  StrictHostKeyChecking no
  User ec2-user

# ansible.cfg
[ssh_connection]
ssh_args = -o ProxyCommand='aws ssm start-session --target %h --document-name AWS-
StartSSHSession --parameters portNumber=22'

# Run playbook — no SSH keys needed
ansible-playbook site.yml -i inventory/prod/
```

Challenge 2 — SSH Host Key Verification at Scale

By default, Ansible refuses to connect to a host whose SSH fingerprint is not in known_hosts. On 100 new servers, this would require accepting each key manually. Solutions:

- Disable host key checking (acceptable for trusted VPC networks):

```
# ansible.cfg
```

```
[defaults]
host_key_checking = False

# Or set per playbook run
ANSIBLE_HOST_KEY_CHECKING=False ansible-playbook site.yml
```

- Pre-populate known_hosts from AWS API (more secure):

```
#!/bin/bash
# Fetch instance public keys and add to known_hosts
for ip in $(aws ec2 describe-instances \
    --filters 'Name=tag:Environment,Values=production' \
    --query 'Reservations[].Instances[].PrivateIpAddress' \
    --output text); do
  ssh-keyscan -H $ip >> ~/.ssh/known_hosts 2>/dev/null
done
```

Challenge 3 — Parallel Execution at Scale

Ansible's default forks=5 means it connects to only 5 hosts at a time. For 100 servers, this serialises execution. Increase forks and use SSH connection optimisations:

```
# ansible.cfg — optimised for 100+ servers
[defaults]
forks              = 50     # Connect to 50 hosts simultaneously
host_key_checking  = False
stdout_callback    = yaml   # Cleaner output at scale

[ssh_connection]
pipelining         = True   # Reduces SSH round trips — significant speedup
ssh_args           = -o ControlMaster=auto -o ControlPersist=60s -o
StrictHostKeyChecking=no
control_path       = /tmp/ansible-ssh-%%r@%%h:%%p
```

Challenge 4 — SSH Connection Failures and Retries

At scale, some servers may be temporarily unreachable (cold start, network blip). Handle this gracefully:

```
# Retry failed hosts
ansible-playbook site.yml -i inventory/prod/
# Creates: site.retry file with unreachable hosts

# Retry only failed hosts
ansible-playbook site.yml -i inventory/prod/ --limit @site.retry

# Or handle within the playbook
- name: Wait for SSH
  ansible.builtin.wait_for:
    host: '{{ inventory_hostname }}'
    port: 22
    delay: 10
    timeout: 120
  delegate_to: localhost
```

Complete Script — Configure 100 Servers

```
#!/bin/bash
# configure-fleet.sh
set -euo pipefail

INVENTORY='inventory/production/'
PLAYBOOK='site.yml'
PRIVATE_KEY='/etc/ansible/ansible_rsa'
LOG_FILE="ansible-run-$(date +%Y%m%d-%H%M%S).log"

echo '=== Pre-flight: Checking connectivity ==='
ansible all -i $INVENTORY -m ping --private-key $PRIVATE_KEY \
  --timeout 10 2>&1 | tee pre-flight.log

REACHABLE=$(grep -c 'SUCCESS' pre-flight.log || true)
TOTAL=$(ansible all -i $INVENTORY --list-hosts 2>/dev/null | tail -n +2 | wc -l)
echo "=== Reachable: $REACHABLE / $TOTAL servers ==="

echo '=== Running playbook ==='
ansible-playbook $PLAYBOOK \
  -i $INVENTORY \
  --private-key $PRIVATE_KEY \
  --forks 50 \
```

```
  --vault-password-file /etc/ansible/.vault_pass \
  -e env=production \
  2>&1 | tee $LOG_FILE

echo '=== Summary ==='
grep -E 'ok=|changed=|failed=' $LOG_FILE | tail -20
```

AWX / Ansible Automation Platform for Enterprise Scale

For organisations running playbooks against hundreds of servers regularly, Red Hat Ansible Automation Platform (AWX open-source) provides:

- Credential vault — SSH keys stored encrypted in a database, injected at runtime. No key files on disk.
- Job templates — Pre-configured playbook runs with RBAC controls.
- Logging and audit trail — Every playbook run logged with host output.
- Dynamic inventory plugins — Real-time server discovery from AWS, Azure, vSphere.
- API-driven execution — Other systems (Terraform, ServiceNow) trigger Ansible runs via REST API.
- Webhook triggers — Git push triggers provisioning playbooks automatically.

Approach	SSH Keys Needed
Standard SSH + key pair	Yes — distribute keys
Cloud-init injection	Yes — at launch time
AWS SSM proxy	No — uses IAM roles
AWX + credential vault	Stored in AWX vault

> *Tip: For the highest security at scale, combine cloud-init for key injection (eliminates bootstrap step) with AWS SSM as the connection proxy (eliminates port 22 exposure). You get zero SSH key files to manage and no open SSH ports — Ansible connects via IAM-authenticated SSM sessions.*
